Frequency-Bottlenecked CycleGAN

Theoretical Prerequisites & Literature Review

CS424 Group Project — Task 2

March 2026

Contents

1 Introduction **2**

2 Mathematical Foundations **2**
- 2.1 Linear Algebra Essentials . . . 2
- 2.2 Multivariable Calculus & Backpropagation . . . 2
- 2.3 Signal Processing & Fourier Analysis . . . 3
- 2.4 Gaussian Filtering & the Frequency Bottleneck . . . 4

3 Deep Learning Theory **5**
- 3.1 Generative Adversarial Networks . . . 5
- 3.2 CycleGAN Architecture . . . 6
- 3.3 Generator Architecture: ResNet-9 Blocks . . . 6
- 3.4 Discriminator Architecture: PatchGAN . . . 7
- 3.5 The Steganography Problem . . . 8
- 3.6 Image Replay Buffer . . . 9

4 Medical Imaging Domain **9**
- 4.1 Brain MRI & FLAIR Sequences . . . 9
- 4.2 BraTS 2020 Dataset . . . 9
- 4.3 NIfTI Data Pipeline . . . 10

5 Evaluation Metrics **11**
- 5.1 Fréchet Inception Distance (FID) . . . 11
- 5.2 Structural Similarity Index (SSIM) . . . 11
- 5.3 Dice Similarity Coefficient . . . 11

6 Forensic Evaluation: Steganographic Audit **12**
- 6.1 FFT Residual Analysis . . . 12
- 6.2 Perturbation Robustness Test . . . 12
- 6.3 Classifier Leakage Test . . . 13

7 Training Practicalities **13**
- 7.1 Optimization . . . 13
- 7.2 Memory Management on T4/P100 . . . 14

8 Literature Review **14**
8.1 Steganography in CycleGAN 14
8.2 Frequency-Domain Analysis of GANs 15
8.3 Mitigating Information Hiding in Cycle-Consistent Models 15
8.4 Medical Image Synthesis with GANs 16
8.5 Alternative Unpaired Translation Methods 16
8.6 Technical Backbone 17

9 Statistical Rigor & Experimental Design **17**
9.1 Multiple Seeds and Reporting 17
9.2 Novelty Framing for Presentation & Report 18

10 Summary: What Each Team Member Must Know **19**

1 Introduction

This document is a self-contained reference covering every theoretical prerequisite needed to implement FB-CycleGAN. It is organized to match the project's architecture: we start with the mathematical foundations (linear algebra, calculus, signal processing), build up to deep learning theory (GANs, cycle-consistency, architectures), cover the medical imaging domain, and conclude with evaluation metrics and a literature review.

How to use this document: Each section ends with edge cases and pitfalls specific to our project. Read the theory, then read the edge cases—those are where projects fail.

2 Mathematical Foundations

2.1 Linear Algebra Essentials

Definition 2.1 (Affine Transformation)**.** Every layer in a neural network without activation computes $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, where $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ is the weight matrix, $\boldsymbol{x} \in \mathbb{R}^n$ is the input, and $\boldsymbol{b} \in \mathbb{R}^m$ is the bias.

For convolutional layers, the "weight matrix" is replaced by a convolution kernel $\boldsymbol{K} \in \mathbb{R}^{c_{\text{out}}\times c_{\text{in}}\times k\times k}$, but the operation is still linear. This matters because:

(a) **Gradient flow** through the network is computed via the chain rule over these linear maps.

(b) **Skip connections** (ResNet blocks) add the identity: $\boldsymbol{y} = \mathcal{F}(\boldsymbol{x}) + \boldsymbol{x}$, which preserves the gradient magnitude (prevents vanishing gradients).

(c) **Transpose convolutions** used in the generator's upsampling path are the adjoint of convolutions. They can introduce checkerboard artifacts if the kernel size is not divisible by the stride.

Edge Case / Pitfall

Transpose convolutions with kernel_size = 4, stride = 2 produce checkerboard artifacts because the output pixels receive uneven contributions from the kernel. The standard CycleGAN uses this configuration but mitigates it via instance normalization. If you switch to nearest-neighbor upsampling + convolution, you avoid the artifact entirely but change the gradient dynamics.

2.2 Multivariable Calculus & Backpropagation

Backpropagation computes $\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_\ell}$ for each layer ℓ via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_\ell} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{a}_L} \cdot \frac{\partial \boldsymbol{a}_L}{\partial \boldsymbol{a}_{L-1}} \cdots \frac{\partial \boldsymbol{a}_{\ell+1}}{\partial \boldsymbol{a}_\ell} \cdot \frac{\partial \boldsymbol{a}_\ell}{\partial \boldsymbol{W}_\ell} \tag{1}$$

where $\boldsymbol{a}_\ell$ is the activation at layer ℓ.

Key Insight for FB-CycleGAN

The Gaussian blur $B(\cdot)$ in FB-CycleGAN is inserted *inside* the computational graph. Because Gaussian blur is a linear convolution with fixed (non-learnable) weights, it is fully differentiable. Gradients flow through B via:

$$\frac{\partial \mathcal{L}_{\text{FB-cyc}}}{\partial G} = \frac{\partial \mathcal{L}}{\partial F(B(G(p)))} \cdot \frac{\partial F}{\partial B(G(p))} \cdot \frac{\partial B}{\partial G(p)} \cdot \frac{\partial G}{\partial \theta_G}$$

The term $\frac{\partial B}{\partial G(p)}$ is just the Gaussian kernel applied to the gradient map (convolution is its own adjoint for symmetric kernels). This means the blur **attenuates high-frequency gradient components** in addition to attenuating high-frequency activations.

Edge Case / Pitfall

Because B attenuates high-frequency gradients, the generator receives weaker learning signal for fine texture details. At large σ, this can cause the generator to produce overly smooth outputs (low FID but also low SSIM). This is the honesty-vs-quality tradeoff you will observe in the Pareto analysis.

2.3 Signal Processing & Fourier Analysis

Definition 2.2 (2D Discrete Fourier Transform)**.** For an image $f[m,n]$ of size $M \times N$:

$$F[u,v] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m,n] \cdot e^{-j2\pi\left(\frac{um}{M} + \frac{vn}{N}\right)} \tag{2}$$

The **power spectrum** is $|F[u,v]|^2$, which decomposes the image into spatial frequency components. Low frequencies (near center after `fftshift`) encode global structure; high frequencies (near edges) encode fine detail and noise.

Why this matters for steganography detection:

When a CycleGAN generator hides information steganographically, it encodes data as imperceptible high-frequency patterns. Given a generated image $\hat{h} = G(p)$ and its source p, the residual $r = |\hat{h} - p|$ should be:

- **Steganographic generator:** Structured high-frequency energy in $|F_r[u,v]|^2$ (information-carrying patterns).
- **Honest generator:** Diffuse, unstructured spectrum (only natural image differences).

Definition 2.3 (Power Spectral Density — Radial Average)**.** To visualize the frequency content as a 1D curve, compute the radially averaged power spectrum:

$$P(\rho) = \frac{1}{|\{(u,v) : \sqrt{u^2+v^2} \approx \rho\}|} \sum_{\sqrt{u^2+v^2} \approx \rho} |F[u,v]|^2 \tag{3}$$

Plot $P(\rho)$ vs. ρ on a log-log scale. Steganographic artifacts appear as bumps or plateaus at high ρ.

Edge Case / Pitfall

When computing FFT residuals, be careful about:

1. **Windowing:** Image boundaries cause spectral leakage. Apply a Hann or Tukey window before FFT.
2. **Normalization:** Compare spectra on the same scale. Normalize by image energy.
3. **Batch statistics:** A single image's spectrum is noisy. Average over many samples to see the steganographic pattern.

2.4 Gaussian Filtering & the Frequency Bottleneck

Definition 2.4 (2D Gaussian Kernel)**.** The continuous 2D Gaussian with standard deviation σ:

$$g(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{4}$$

Discretized to a $k \times k$ kernel. The relationship between k and σ: typically $k = \lceil 6\sigma \rceil$ rounded to odd, ensuring $> 99.7\%$ of the Gaussian mass is captured.

Definition 2.5 (Low-Pass Filter Interpretation)**.** The Fourier transform of a Gaussian is a Gaussian:

$$\mathcal{F}\{g\}(u, v) = \exp\left(-2\pi^2\sigma^2(u^2 + v^2)\right) \tag{5}$$

This means $B(\cdot)$ attenuates frequency component (u, v) by a factor of $\exp(-2\pi^2\sigma^2(u^2 + v^2))$. The **effective cutoff frequency** (where attenuation reaches $1/e$) is:

$$f_{\text{cutoff}} = \frac{1}{2\pi\sigma} \tag{6}$$

Key Insight for FB-CycleGAN

This is the core mechanism of FB-CycleGAN. The steganographic channel capacity is bounded by the information that can pass through B. By choosing σ, you control the maximum spatial frequency available for encoding. A larger σ means a lower cutoff frequency, which means less bandwidth for hiding information, but also less capacity for preserving legitimate high-frequency detail (edges, texture).

Information-theoretic view: The blur acts as a channel with limited bandwidth. The channel capacity C scales inversely with σ. At sufficiently large σ, the capacity is too low to encode the tumor coordinates needed for cycle reconstruction, forcing the generator to preserve pathology visibly.

Edge Case / Pitfall

Kernel size vs. sigma mismatch: If k is too small relative to σ, the kernel is truncated and the actual filter is not a true Gaussian. This introduces ringing artifacts. Always use $k \geq 6\sigma + 1$.

Boundary effects: `torchvision.transforms.GaussianBlur` uses reflection padding by default. This is fine for natural images but can cause artifacts at brain boundaries where the image transitions to zero background. Consider padding with zeros or the image mean before blurring.

Gradient magnitude: The blur reduces gradient norms by $\approx 1/\sigma^2$ for high-frequency components. At $\sigma = 3$, the generator gets $\sim 9\times$ weaker gradients for fine details. You may need to adjust learning rates or use gradient scaling.

3 Deep Learning Theory

3.1 Generative Adversarial Networks

Definition 3.1 (GAN Minimax Objective)**.** The original GAN objective (Goodfellow et al., 2014):

$$\min_G \max_D \; \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_z}[\log(1 - D(G(\boldsymbol{z})))] \tag{7}$$

At the Nash equilibrium, $D^*(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x})+p_G(\boldsymbol{x})} = \frac{1}{2}$ and G generates samples from p_{data}.

Training instabilities:

1. **Vanishing gradients:** When D is too strong, $D(G(\boldsymbol{z})) \approx 0$ for all generated samples, and $\log(1 - D(G(\boldsymbol{z}))) \approx 0$ provides near-zero gradient to G.
2. **Mode collapse:** G maps all inputs to a single output that fools D, ignoring the diversity of the data distribution.
3. **Oscillation:** G and D chase each other without converging.

Definition 3.2 (LSGAN — Least Squares GAN)**.** Mao et al. (2017) replace the log-likelihood with L2 loss:

$$\mathcal{L}_D = \frac{1}{2}\mathbb{E}_{\boldsymbol{x}}[(D(\boldsymbol{x}) - 1)^2] + \frac{1}{2}\mathbb{E}_{\boldsymbol{z}}[D(G(\boldsymbol{z}))^2] \tag{8}$$

$$\mathcal{L}_G = \frac{1}{2}\mathbb{E}_{\boldsymbol{z}}[(D(G(\boldsymbol{z})) - 1)^2] \tag{9}$$

LSGAN penalizes samples that are far from the decision boundary even if they are on the correct side, providing non-vanishing gradients. This is the default in CycleGAN.

Edge Case / Pitfall

LSGAN target values: The original LSGAN paper uses $a = 0, b = 1, c = 1$ (real=1, fake=0, generator target=1). Some implementations use $a = -1, b = 1, c = 0$ (centered). Both work but you must be consistent. Check your implementation matches the discriminator's output range.

3.2 CycleGAN Architecture

CycleGAN (Zhu et al., 2017) learns two mappings $G : \mathcal{P} \to \mathcal{H}$ and $F : \mathcal{H} \to \mathcal{P}$ with two discriminators D_H and D_P.

Full objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GAN}}(G, D_H) + \mathcal{L}_{\text{GAN}}(F, D_P) + \lambda_{\text{cyc}}\mathcal{L}_{\text{cyc}}(G, F) + \lambda_{\text{id}}\mathcal{L}_{\text{id}}(G, F) \tag{10}$$

Cycle-consistency loss:

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_p\left[\|F(G(p)) - p\|_1\right] + \mathbb{E}_h\left[\|G(F(h)) - h\|_1\right] \tag{11}$$

Identity loss:

$$\mathcal{L}_{\text{id}}(G, F) = \mathbb{E}_h\left[\|G(h) - h\|_1\right] + \mathbb{E}_p\left[\|F(p) - p\|_1\right] \tag{12}$$

FB-CycleGAN modification: Replace $\mathcal{L}_{\text{cyc}}$ with:

$$\mathcal{L}_{\text{FB-cyc}}(G, F) = \mathbb{E}_p\left[\|F(\underbrace{B(G(p))}_{\text{blurred}}) - p\|_1\right] + \mathbb{E}_h\left[\|G(\underbrace{B(F(h))}_{\text{blurred}}) - h\|_1\right] \tag{13}$$

Key Insight for FB-CycleGAN

The identity loss $\mathcal{L}_{\text{id}}$ is weighted by $\lambda_{\text{id}} \cdot \lambda_{\text{cyc}}$ (i.e., `lambda_identity` $\times$ `lambda_cycle` $= 0.5 \times 10 = 5$). This is a strong regularizer. In the FB-CycleGAN setting, you may want to experiment with reducing λ_{id} since the bottleneck already provides structural regularization.

Edge Case / Pitfall

Where exactly to insert the blur. The blur goes between G's output and F's input in the cycle path, and between F's output and G's input. It does **not** go:

- Before the discriminator (that would change what D sees and break adversarial training).
- In the identity loss path (identity loss compares $G(h)$ to h directly, no cycle involved).
- In the forward pass for generating final outputs at test time (blur is training-only in the cycle path).

Getting this wrong fundamentally changes the method.

3.3 Generator Architecture: ResNet-9 Blocks

The standard CycleGAN generator for 256×256 images:

1. **Encoder** (downsampling):
   - 7×7 Conv, InstanceNorm, ReLU ($c_{\text{in}} \to 64$) with ReflectionPad
   - 3×3 Conv stride 2, InstanceNorm, ReLU ($64 \to 128$)
   - 3×3 Conv stride 2, InstanceNorm, ReLU ($128 \to 256$)
2. **Transformer** (residual blocks):

- 9× ResNet block: ReflectionPad → 3×3 Conv → InstanceNorm → ReLU → ReflectionPad → 3×3 Conv → InstanceNorm + skip

3. **Decoder** (upsampling):
   - 3×3 ConvTranspose stride 2, InstanceNorm, ReLU ($256 \to 128$)
   - 3×3 ConvTranspose stride 2, InstanceNorm, ReLU ($128 \to 64$)
   - 7×7 Conv, Tanh ($64 \to c_{\text{out}}$) with ReflectionPad

Definition 3.3 (Instance Normalization)**.** Unlike BatchNorm (normalizes across the batch), InstanceNorm normalizes each sample and channel independently:

$$\text{IN}(x_{n,c}) = \frac{x_{n,c} - \mu_{n,c}}{\sqrt{\sigma_{n,c}^2 + \epsilon}} \tag{14}$$

where $\mu_{n,c}$ and $\sigma_{n,c}^2$ are computed over the spatial dimensions $H \times W$ for each sample n and channel c. This is crucial for style transfer as it removes instance-specific contrast information.

Edge Case / Pitfall

InstanceNorm with single-channel input: For our 1-channel FLAIR MRI, InstanceNorm normalizes across the entire 256×256 spatial extent of each image. This effectively removes the global intensity information from each feature map. For MRI, where absolute intensity carries diagnostic information, this could be problematic. Consider:

- Using InstanceNorm with `affine=True` to learn a scale/shift
- Or experimenting with GroupNorm or LayerNorm as alternatives

Edge Case / Pitfall

ReflectionPad vs. ZeroPad: ReflectionPad avoids border artifacts by mirroring the image at boundaries. However, for brain MRI where the brain is surrounded by zero background, reflection padding at the brain-background boundary reflects actual brain content into the background. This is usually acceptable but worth monitoring in generated outputs.

3.4 Discriminator Architecture: PatchGAN

Definition 3.4 (PatchGAN)**.** Instead of classifying the entire image as real/fake, PatchGAN outputs an $N \times N$ map where each element classifies a **receptive field patch** of the input. The 70×70 PatchGAN:

1. 4×4 Conv stride 2, LeakyReLU(0.2) ($c_{\text{in}} \to 64$)
2. 4×4 Conv stride 2, InstanceNorm, LeakyReLU(0.2) ($64 \to 128$)
3. 4×4 Conv stride 2, InstanceNorm, LeakyReLU(0.2) ($128 \to 256$)
4. 4×4 Conv stride 1, InstanceNorm, LeakyReLU(0.2) ($256 \to 512$)
5. 4×4 Conv stride 1 ($512 \to 1$)

Receptive field calculation:

$$r_\ell = r_{\ell-1} + (k_\ell - 1) \cdot \prod_{i=1}^{\ell-1} s_i \tag{15}$$

where r_ℓ is the receptive field at layer ℓ, k_ℓ is the kernel size, and s_i is the stride at layer i.

For the 70×70 PatchGAN:

$$r_1 = 4 \tag{16}$$
$$r_2 = 4 + (4-1) \cdot 2 = 10 \tag{17}$$
$$r_3 = 10 + (4-1) \cdot 4 = 22 \tag{18}$$
$$r_4 = 22 + (4-1) \cdot 8 = 46 \tag{19}$$
$$r_5 = 46 + (4-1) \cdot 8 = 70 \quad \checkmark \tag{20}$$

Key Insight for FB-CycleGAN

The 70×70 receptive field means the discriminator focuses on **local texture** rather than global structure. This is good for our task: we want the generator to produce locally realistic tissue textures. However, it also means the discriminator cannot detect global structural inconsistencies (e.g., a tumor appearing in an anatomically impossible location). The cycle-consistency loss handles global structure instead.

3.5 The Steganography Problem

Why does steganography happen? Consider the forward cycle: $p \xrightarrow{G} \hat{h} \xrightarrow{F} \hat{p}$. The cycle loss penalizes $\|\hat{p} - p\|_1$. If domains $\mathcal{P}$ (pathological) and $\mathcal{H}$ (healthy) are structurally asymmetric (one has tumors, the other doesn't), then G has two options:

1. **Honest translation:** Remove the tumor from p to produce $\hat{h}$. But then F must reconstruct the tumor from $\hat{h}$, which is impossible without knowing where the tumor was. High $\mathcal{L}_{\text{cyc}}$.

2. **Steganographic encoding:** G produces an image $\hat{h}$ that *looks* healthy but encodes the tumor's location and shape as imperceptible high-frequency noise. F reads this noise to perfectly reconstruct p. Low $\mathcal{L}_{\text{cyc}}$.

Option 2 achieves lower training loss, so gradient descent converges to it. The steganographic channel capacity of a 256×256 image is enormous—even 1 bit per pixel gives 65536 bits, more than enough to encode a tumor mask.

Key Insight for FB-CycleGAN

FB-CycleGAN attacks this by reducing the channel capacity. After $B(\cdot)$ with cutoff frequency $f_c = \frac{1}{2\pi\sigma}$, the effective number of independent frequency components that survive is:

$$N_{\text{eff}} \approx \pi \left(\frac{N}{2\pi\sigma} \right)^2 = \frac{N^2}{4\pi\sigma^2}$$

For $N = 256$, $\sigma = 1$: $N_{\text{eff}} \approx 5215$ (still large). For $\sigma = 3$: $N_{\text{eff}} \approx 579$ (severely restricted). At some σ^*, the capacity drops below what's needed to encode the tumor, forcing honest translation.

Edge Case / Pitfall

Partial steganography: The generator might learn to encode *some* information in the passband frequencies (below the cutoff). The blur doesn't eliminate *all* information hiding—it only eliminates the high-frequency channel. Monitor the low-frequency residual spectrum for structured patterns even with the bottleneck active.

3.6 Image Replay Buffer

Definition 3.5 (Replay Buffer for GANs)**.** Maintain a buffer of N (typically 50) previously generated images. When updating the discriminator, with probability 0.5, replace the current generated image with a random image from the buffer (and add the current one to the buffer). This prevents the discriminator from overfitting to the generator's most recent outputs and stabilizes training.

Edge Case / Pitfall

Memory management: With batch size 1 and 256×256 single-channel images, a 50-image buffer is ∼12.5 MB (negligible). But if you experiment with batch size > 1 or multi-channel inputs, the buffer scales as $B \times C \times H \times W \times 4$ bytes $\times$ 50.

4 Medical Imaging Domain

4.1 Brain MRI & FLAIR Sequences

Magnetic Resonance Imaging (MRI) produces 3D volumetric scans by measuring the magnetic relaxation properties of hydrogen atoms in tissue. Different **pulse sequences** weight different tissue properties:

Sequence	Weighting	Clinical Use
T1	Longitudinal relaxation	Anatomy, gray/white matter contrast
T1CE	T1 + gadolinium contrast	Enhancing tumor boundaries
T2	Transverse relaxation	Edema, fluid-filled structures
FLAIR	**T2 with CSF suppression**	**Tumor + edema (highest contrast)**

Why FLAIR: In T2-weighted images, both tumor edema and cerebrospinal fluid (CSF) appear bright, making it hard to distinguish them. FLAIR suppresses the CSF signal, so only pathological tissue remains bright. This gives the highest tumor-to-background contrast, which is why the proposal uses FLAIR.

4.2 BraTS 2020 Dataset

- 369 multi-parametric MRI volumes: 293 HGG (high-grade glioma) + 76 LGG (low-grade glioma)
- Each volume: 4 sequences (T1, T1CE, T2, FLAIR) at $240 \times 240 \times 155$ voxels
- Voxel-level segmentation mask: enhancing tumor (ET), peritumoral edema (ED), necrotic core (NCR)
- Stored as NIfTI (`.nii.gz`) files

Domain assignment per 2D axial slice:

- $\mathcal{P}$ (pathological): segmentation mask shows tumor area > 5% of brain region
- $\mathcal{H}$ (healthy): mask is entirely zero AND brain area > minimum threshold

Edge Case / Pitfall

Slice selection bias: Tumors are concentrated in the middle axial slices. The top and bottom slices are mostly empty skull/air. If you naively extract all 155 slices, you'll have a massive class imbalance (many empty or near-empty slices in $\mathcal{H}$). Filter aggressively: require minimum brain area in healthy slices.

Inter-slice correlation: Adjacent slices from the same volume are highly correlated. If you split train/val/test by slice rather than by patient, you'll have severe data leakage. **Always split by patient ID.**

Intensity non-uniformity: MRI intensities are not standardized across scanners or patients. The proposal specifies Z-score normalization **per volume** (not per slice). This is important—per-slice normalization would destroy the relative intensity information within a volume.

4.3 NIfTI Data Pipeline

Loading a NIfTI volume with nibabel:

1. Load: `nib.load(path)` returns a NIfTI image object
2. Extract array: `np.asarray(img.dataobj)` $\rightarrow$ shape $(240, 240, 155)$
3. Extract axial slices: `volume[:, :, z]` for slice index z
4. The segmentation mask has values: 0 (background), 1 (NCR), 2 (ED), 4 (ET)

Preprocessing per slice:

1. Compute brain mask: `brain = volume > 0`
2. Find bounding box of brain region
3. Center-crop to bounding box
4. Resize to 256×256 (bilinear interpolation for images, nearest-neighbor for masks)
5. Z-score normalize using volume-level statistics: $x' = \frac{x - \mu_{\text{vol}}}{\sigma_{\text{vol}}}$ (computed only over non-zero voxels)

Edge Case / Pitfall

Mask label encoding: BraTS uses labels $\{0, 1, 2, 4\}$ (note: no label 3). When computing tumor area, use mask > 0. Don't assume contiguous labels.

Resize interpolation: Using bilinear interpolation on the segmentation mask will create non-integer values at boundaries. Always use nearest-neighbor for masks.

5 Evaluation Metrics

5.1 Fréchet Inception Distance (FID)

Definition 5.1 (FID)**.** FID measures the distance between the real and generated image distributions in the feature space of an Inception-v3 network:

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g)^{1/2}\right) \tag{21}$$

where $(\boldsymbol{\mu}_r, \boldsymbol{\Sigma}_r)$ and $(\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g)$ are the mean and covariance of Inception features for real and generated images respectively. Lower FID = better.

Edge Case / Pitfall

FID on single-channel medical images: Inception-v3 expects 3-channel RGB input. For our 1-channel FLAIR images, you must either:

- Replicate the channel 3 times: $x_{\text{rgb}} = [x, x, x]$
- Or use a medical-image-specific feature extractor (e.g., a pretrained ResNet on RadImageNet)

The first approach is standard but the FID values won't be comparable to those reported on natural image benchmarks. Be consistent across your own experiments.

Sample size: FID is biased for small sample sizes. Use at least 500 images from each distribution, ideally more. Report the number of samples used.

5.2 Structural Similarity Index (SSIM)

Definition 5.2 (SSIM)**.** SSIM compares images on three components—luminance (l), contrast (c), and structure (s):

$$\text{SSIM}(x, y) = l(x, y) \cdot c(x, y) \cdot s(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{22}$$

where $C_1 = (K_1L)^2$, $C_2 = (K_2L)^2$ are stabilization constants, L is the dynamic range, and statistics are computed over local windows.

In our context: SSIM measures cycle-reconstruction fidelity: how well does $F(B(G(p)))$ match p? Higher SSIM = better reconstruction. The honesty-quality tradeoff: at high σ, the blur destroys too much information for accurate reconstruction (low SSIM), but the generator is more honest.

Edge Case / Pitfall

Dynamic range: If your images are normalized to $[-1, 1]$ (Tanh output), set `data_range=2.0`. If normalized to $[0, 1]$, set `data_range=1.0`. Getting this wrong gives meaningless SSIM values.

5.3 Dice Similarity Coefficient

Definition 5.3 (Dice Score)**.** For binary segmentation masks P (prediction) and G (ground truth):

$$\text{Dice}(P, G) = \frac{2|P \cap G|}{|P| + |G|} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \tag{23}$$

Dice $\in [0, 1]$, where 1 is perfect overlap. It is the harmonic mean of precision and recall.

In our context: A U-Net is trained on synthetic pathological images from CycleGAN and evaluated on real BraTS data. Higher Dice from FB-CycleGAN synthetic data (vs. baseline) confirms that the frequency bottleneck improves downstream clinical utility.

Edge Case / Pitfall

Multi-class Dice: BraTS has three tumor sub-regions (ET, ED, NCR). Report Dice per class and the mean. Enhancing tumor (ET) is typically the hardest class—small, variable shape.

Empty predictions: When both P and G are empty (true negative slice), Dice is undefined (0/0). Convention: set Dice = 1.0 for true negatives, 0.0 otherwise.

6 Forensic Evaluation: Steganographic Audit

6.1 FFT Residual Analysis

For each generated image $\hat{h} = G(p)$:

1. Compute residual: $r = |\hat{h} - p|$
2. Apply windowing: $r_w = r \cdot w_{\text{Hann}}$ (to reduce spectral leakage)
3. Compute 2D FFT: $R = \text{FFT2}(r_w)$
4. Compute power spectrum: $S = |R|^2$
5. Shift zero-frequency to center: $S_c = \text{fftshift}(S)$
6. Compute radial average $P(\rho)$ and plot log-log
7. Average over many samples for statistical significance

What to look for:

- **Baseline CycleGAN:** Peaks or plateaus at high frequencies (steganographic encoding)
- **FB-CycleGAN:** Steep rolloff at high frequencies, no structured peaks

6.2 Perturbation Robustness Test

Inject additive Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$ into the generated image before feeding it to the reverse generator:

$$\hat{p}_{\text{perturbed}} = F(\hat{h} + \epsilon) \tag{24}$$

Measure the L1 reconstruction error $\|\hat{p}_{\text{perturbed}} - p\|_1$ as a function of σ_ϵ.

- **Steganographic generator:** Catastrophic degradation ($> 50\%$ L1 increase) because the noise destroys the encoded information.
- **Honest generator:** Graceful degradation because the reconstruction relies on visible structural features, not hidden signals.

Edge Case / Pitfall

Noise scale: The perturbation σ_ϵ must be small enough that it wouldn't destroy visible features in a natural image, but large enough to destroy steganographic patterns. Start with $\sigma_\epsilon \in \{0.001, 0.005, 0.01, 0.02, 0.05\}$ and plot the degradation curve.

6.3 Classifier Leakage Test

Beyond spectral forensics, an **active adversarial test** provides direct evidence of information hiding.

Definition 6.1 (Classifier Leakage Test)**.** Train a lightweight classifier (e.g., ResNet-18) on the generated "healthy" images $\hat{h} = G(p)$ to predict characteristics of the original pathological input p (e.g., tumor presence, tumor size bin, or tumor location quadrant). If the classifier achieves accuracy significantly above chance, the generated images contain encoded information about the source—proof of steganography.

Protocol:

1. Generate healthy images $\hat{h} = G(p)$ for all pathological inputs p in the test set.
2. Label each $\hat{h}$ with the original tumor characteristics (from the segmentation mask of p).
3. Train a ResNet-18 classifier: $\hat{h} \to$ tumor property.
4. **Baseline CycleGAN:** Expect high accuracy (information is hidden in $\hat{h}$).
5. **FB-CycleGAN:** Expect accuracy $\approx$ random chance (information destroyed by blur).

Key Insight for FB-CycleGAN

This test is more convincing than FFT analysis alone because it directly measures *usable* hidden information rather than just spectral artifacts. A reviewer can look at your FFT plots and argue about whether the patterns are "structured enough" to constitute steganography. A classifier that can recover tumor location from "healthy" images is unambiguous proof.

Edge Case / Pitfall

What to classify: Start with binary (tumor present vs. not)—but this is trivial since all $\hat{h}$ come from pathological inputs. Better targets:

- Tumor **location** (which quadrant of the brain)
- Tumor **size** (small/medium/large bins)
- Tumor **type** (HGG vs. LGG)

If the classifier can recover *any* of these from the "healthy" output, steganography is confirmed.

Train/test split: Use the same patient-level split as your main experiment. The classifier must be evaluated on patients unseen during its own training.

7 Training Practicalities

7.1 Optimization

Standard CycleGAN training protocol:

- Optimizer: Adam with $\beta_1 = 0.5$, $\beta_2 = 0.999$ (lower β_1 than default reduces momentum, helpful for GAN stability)
- Learning rate: 2×10^{-4} for both G and D

- Schedule: constant for first 100 epochs, then linearly decay to 0 over next 100 epochs
- Batch size: 1 (instance-level training, paired with InstanceNorm)

Edge Case / Pitfall

Separate optimizers: CycleGAN uses separate optimizers for generators and discriminators. When updating generators, **do not** update discriminator weights, and vice versa. In PyTorch, this means calling `optimizer_G.step()` and `optimizer_D.step()` in separate backward passes, with appropriate `set_requires_grad()` calls to avoid unnecessary gradient computation.

Generator vs. discriminator update ratio: Standard CycleGAN updates both once per iteration (1:1 ratio). If the discriminator dominates, consider updating G more often (2:1 or 3:1).

7.2 Memory Management on T4/P100

Component	Params	Memory (float32)
Generator G ($1 \to 1$, ngf=64)	~11.4M	~43 MB
Generator F ($1 \to 1$, ngf=64)	~11.4M	~43 MB
Discriminator D_H (ndf=64)	~2.8M	~11 MB
Discriminator D_P (ndf=64)	~2.8M	~11 MB
Total parameters	~28.4M	~108 MB
Activations (batch=1, 256×256)	—	~2–4 GB
Optimizer states (Adam, 2×)	—	~216 MB
Total estimated		~3–5 GB

T4 has 16 GB, P100 has 16 GB. You have comfortable headroom at batch size 1.

Edge Case / Pitfall

The cycle path doubles memory: Computing $F(B(G(p)))$ requires keeping $G(p)$ and $B(G(p))$ in memory while F runs. This means the full forward pass allocates activations for both generators simultaneously. Use `torch.cuda.empty_cache()` between generator and discriminator updates if you see OOM.

Gradient accumulation for effective batch size: If you want to simulate batch size > 1 without OOM, accumulate gradients over multiple forward passes before calling `optimizer.step()`.

8 Literature Review

The proposal cites four foundational works. Here we survey additional literature organized by topic, highlighting works directly relevant to the FB-CycleGAN approach.

8.1 Steganography in CycleGAN

Chu, Zhmoginov & Sandler (2017) first demonstrated that CycleGAN hides information steganographically, calling it "a master of steganography." This was expanded by several follow-up works:

- **Bashkirova, Usman & Saenko (2019)** — "Adversarial Self-Defense for Cycle-Consistent GANs." *NeurIPS 2019.* Proposed adversarial noise injection into the cycle path to prevent steganographic encoding. Most directly comparable approach to FB-CycleGAN (stochastic perturbation vs. our deterministic filtering).
- **Zhang et al. (2019)** — "Harmonic Unpaired Image-to-Image Translation" (HarmonicGAN) addressed the information hiding problem by explicitly penalizing high-frequency content in generated images. This is perhaps the closest prior work to FB-CycleGAN but uses a learnable spectral penalty rather than a fixed bottleneck.

8.2 Frequency-Domain Analysis of GANs

- **Durall et al. (2020)** — "Watch Your Up-Convolution: CNN Based Generative Deep Neural Networks are Failing to Reproduce Spectral Distributions." *CVPR 2020.* Showed that GAN-generated images have characteristic spectral artifacts (frequency dropoff). This is relevant because it means even "honest" generators will have spectral signatures.
- **Dzanic et al. (2020)** — "Fourier Spectrum Discrepancies in Deep Network Generated Images." *NeurIPS 2020.* Further characterized the spectral gap between real and generated images.
- **Frank et al. (2020)** — "Leveraging Frequency Analysis for Deep Fake Image Recognition." *ICML 2020.* Used spectral analysis to detect GAN-generated images—the same tools we use for steganographic forensics.
- **Chandrasegaran et al. (2021)** — "A Closer Look at Fourier Spectrum Discrepancies for CNN-generated Images Detection." *CVPR 2021.* Deeper analysis of why frequency artifacts arise from transposed convolutions.
- **Rahaman et al. (2019)** — "On the Spectral Bias of Neural Networks." *ICML 2019.* Foundational paper showing neural networks learn low-frequency components first. Provides theoretical backing for why a low-pass bottleneck still allows meaningful translation while blocking steganographic high-frequency encoding.
- **Jiang et al. (2021)** — "Focal Frequency Loss for Image Reconstruction and Synthesis." *ICCV 2021.* A loss function operating in the frequency domain to improve image generation quality. Could be combined with or compared against the fixed Gaussian bottleneck as a learnable alternative.

8.3 Mitigating Information Hiding in Cycle-Consistent Models

- **Bashkirova, Usman & Saenko (2019)** — "Adversarial Self-Defense for Cycle-Consistent GANs" proposed adding noise to the translated image before the reverse cycle, forcing robustness. Similar motivation to our blur bottleneck but using stochastic perturbation rather than deterministic filtering.
- **Li et al. (2020)** — "Noise-Aware CycleGAN" introduced noise injection in the cycle path specifically to combat steganographic encoding, with analysis of how noise level affects generation quality.
- **Kang et al. (2021)** — "SteganoGAN detection via frequency analysis" provided forensic tools for detecting steganographic encoding in GAN outputs.
- **Porav, Sherrat & Newman (2019)** — "Reducing Steganography in Cycle-Consistency Loss." Directly addressed steganographic encoding in CycleGAN with frequency-aware regularization. **Critical citation** — establishes that the problem space is studied.

- **Baur et al. (2020)** — "SteGANomaly: Inhibiting CycleGAN Steganography for Unsupervised Anomaly Detection in Brain MRI." Applied steganography mitigation specifically to brain MRI anomaly detection. **Most directly related prior work.**
- **Utz (2023)** — Investigated frequency-domain constraints for preventing information hiding in cycle-consistent models. Relevant for positioning FB-CycleGAN's novelty.

8.4 Medical Image Synthesis with GANs

- **Cohen, Luck & Honari (2018)** — Already cited. Showed that distribution matching losses hallucinate features in medical image translation—the foundational motivation for our work.
- **Welander, Karlsson & Eklund (2018)** — "Generative Adversarial Networks for Image-to-Image Translation on Multi-Contrast MR Images" applied CycleGAN to brain MRI modality transfer (T1↔T2), demonstrating both successes and failure modes.
- **Shin et al. (2018)** — "Medical Image Synthesis for Data Augmentation and Anonymization using Generative Adversarial Networks" evaluated GANs for augmenting brain MRI datasets for segmentation training.
- **Yu et al. (2019)** — "EA-GANs: Edge-Aware GANs for Cross-Modality MR Image Synthesis." *IEEE TMI 2019.* Used edge-aware losses to preserve structural boundaries in brain MRI translation—relevant as an alternative approach to preserving pathology.
- **Dar et al. (2019)** — "Image Synthesis in Multi-Contrast MRI with Conditional Generative Adversarial Networks." *IEEE TMI 2019.* Conditional GAN methods for multi-contrast MRI synthesis; demonstrates the importance of faithful structural preservation in medical contexts.
- **Armanious et al. (2020)** — "MedGAN: Medical Image Translation Using GANs." *Computerized Medical Imaging and Graphics 2020.* CascadeNet-based medical image translation with perceptual loss; relevant comparison for non-cycle-consistent medical synthesis.

8.5 Alternative Unpaired Translation Methods

These methods avoid or reformulate cycle-consistency, sidestepping the steganography problem by design. Important to discuss in related work and Q&A.

- **Park et al. (2020)** — "Contrastive Learning for Unpaired Image-to-Image Translation (CUT)." *ECCV 2020.* Replaces cycle-consistency with a contrastive loss that maximizes mutual information between corresponding patches. **Key comparison:** CUT has no reverse generator and no cycle path, so steganography cannot occur by construction. However, it also cannot enforce bidirectional consistency. Discuss why cycle-consistency is still desirable for medical imaging (bidirectional translation needed for data augmentation in both directions).
- **Liu, Breuel & Kautz (2017)** — "Unsupervised Image-to-Image Translation Networks (UNIT)." *NeurIPS 2017.* Assumes a shared latent space between domains. The shared encoder bottleneck limits information flow (similar spirit to our frequency bottleneck but in latent space rather than pixel space).
- **Johnson, Alahi & Fei-Fei (2016)** — "Perceptual Losses for Real-Time Style Transfer and Super-Resolution." *ECCV 2016.* Foundational paper on perceptual (VGG feature-matching) loss. Often combined with GAN losses to reduce hallucination. The proposal

mentions that existing mitigations "rely on heavyweight auxiliary networks (VGG perceptual losses)"—this is the paper being referenced.

Key Insight for FB-CycleGAN

When asked "why not just use CUT instead of CycleGAN + bottleneck?", your answer: (1) CUT is one-directional by design—we need both P→H and H→P for data augmentation. (2) The cycle-consistency framework with our bottleneck gives us a principled knob (σ) to control the honesty-quality tradeoff. (3) Comparing baseline CycleGAN vs. FB-CycleGAN isolates the effect of the bottleneck, which is cleaner experimentally than comparing entirely different architectures.

8.6 Technical Backbone

- **Goodfellow et al. (2014)** — "Generative Adversarial Networks." The original GAN paper.
- **Mao et al. (2017)** — "Least Squares Generative Adversarial Networks." LSGAN objective used in CycleGAN.
- **Isola et al. (2017)** — "Image-to-Image Translation with Conditional Adversarial Networks" (pix2pix). Introduced PatchGAN discriminator used in CycleGAN.
- **Ulyanov, Vedaldi & Lempitsky (2016)** — "Instance Normalization: The Missing Ingredient for Fast Stylization." InstanceNorm is critical for style transfer quality.
- **He et al. (2016)** — "Deep Residual Learning for Image Recognition." ResNet skip connections used in the generator.
- **Ronneberger, Fischer & Brox (2015)** — "U-Net: Convolutional Networks for Biomedical Image Segmentation." Architecture for downstream segmentation evaluation.
- **Heusel et al. (2017)** — "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium." *NeurIPS 2017.* Introduced FID.
- **Wang et al. (2004)** — "Image Quality Assessment: From Error Visibility to Structural Similarity." *IEEE TIP 2004.* Introduced SSIM. Both FID and SSIM are primary evaluation metrics.
- **Shrivastava et al. (2017)** — "Learning from Simulated and Unsupervised Images through Adversarial Training (SimGAN)." *CVPR 2017.* Introduced the image replay buffer technique adopted by CycleGAN for stabilizing discriminator training.

9 Statistical Rigor & Experimental Design

9.1 Multiple Seeds and Reporting

Single-run results in deep learning are unreliable. GAN training is particularly sensitive to initialization and data ordering.

Mandatory protocol:

- Train each configuration across **at least 3 random seeds**
- Report all metrics as Mean $\pm$ Std (e.g., "FID: 42.3 ± 1.7")

- For the downstream U-Net evaluation: 3 seeds for U-Net training × each CycleGAN variant
- Use statistical tests (paired t-test or Wilcoxon signed-rank) to claim one method is "better" than another

Edge Case / Pitfall

Seed management: Set seeds for Python, NumPy, PyTorch, and CUDA. But note that some PyTorch operations are non-deterministic on GPU even with fixed seeds (e.g., `torch.nn.functional.interpolate` with `mode='bilinear'`). Set `torch.backends.cudnn.deterministic = True` and `torch.backends.cudnn.benchmark = False` for maximum reproducibility, but accept that perfect reproducibility across different hardware is not guaranteed.

Compute budget: 3 seeds × (1 baseline + 5 sigma variants) × 200 epochs = 3600 training epochs total. On a T4 with 256×256 single-channel images, estimate ∼2–5 min per epoch. Total: ∼120–300 GPU-hours. Plan accordingly with Kaggle/Colab time limits.

9.2 Novelty Framing for Presentation & Report

Frequency-based steganography mitigation in GANs is **not** unstudied. Prior works include noise injection (Bashkirova et al., 2019), spectral penalties (HarmonicGAN, Zhang et al., 2019), and steganography-aware training (Porav et al., 2019; Baur et al., SteGANomaly, 2020; Utz, 2023).

Your actual novelty claims (defensible during Q&A):

1. **Application context:** Applying a frequency bottleneck to the severe structural asymmetry of BraTS pathological ↔ healthy brain MRI, where the steganographic incentive is strongest.
2. **Systematic σ Pareto analysis:** Mapping the exact quantitative tradeoff between translation honesty and generation quality across multiple blur strengths—not just "blur helps" but "here is the optimal operating point."
3. **Dual-pronged forensic audit:** Combining spectral analysis (FFT residuals) with the classifier leakage test for rigorous steganography detection.
4. **Downstream clinical validation:** Showing that the bottleneck improves not just forensic metrics but actual segmentation performance on real data.

Edge Case / Pitfall

During Q&A: If asked "how is this different from just adding noise to the cycle path?", the answer is: (1) Gaussian blur is deterministic and targets a specific frequency band, giving precise control via σ; noise injection is stochastic and affects all frequencies equally. (2) Blur preserves low-frequency structure exactly while destroying high-frequency content; noise degrades everything. (3) The Pareto sweep over σ gives an interpretable honesty-quality curve; noise injection has no equivalent principled sweep.

10 Summary: What Each Team Member Must Know

Models & Architecture	**Data & Evaluation**	**Training & Infra**
ResNet-9 block structure	NIfTI parsing with nibabel	Adam optimizer ($\beta_1 = 0.5$)
PatchGAN 70×70 RF calc	Axial slice extraction	Linear LR decay schedule
InstanceNorm (affine=True?)	Domain filtering (5% threshold)	Replay buffer (pool=50)
Transpose conv artifacts	Z-score normalization	Separate G/D optimizers
Gaussian kernel math	FID (single-channel handling)	`set_requires_grad()`
Frequency cutoff $\frac{1}{2\pi\sigma}$	SSIM (data_range!)	W&B sweep logging
Blur placement in cycle path	Dice (multi-class, edge cases)	Gradient accumulation
All: Sections 2–3 of this doc	FFT residual + perturbation test	Memory budgeting

Before Writing Code

1. Every team member reads Sections 1–4 (mathematical foundations through evaluation)
2. Each person deep-dives into their column above
3. Run `scripts/analyze_data.py` together so everyone sees the data
4. Discuss edge cases from this document as a group
5. Only then start implementing